

Embedded System Lab Manual

LAB EXPERIMENT NO. 2

Project Setup and Blinky & Running LEDs C++

Submitted to :

Mr Yasir

Submitted by:

Ahsan Basharat

Date of Submission:

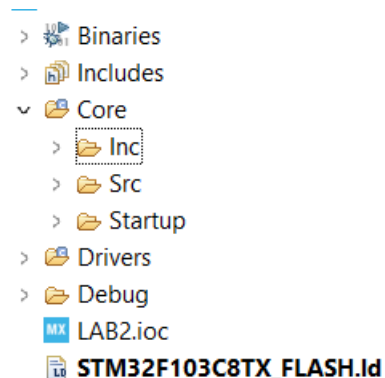
2/07/2027

Introduction

In this lab, we learned the basic project structure of STM32CubeIDE, including the purpose of important folders and files such as Core, Drivers, Includes, and the .ioc configuration file. We also developed a simple LED blinking program using C/C++ and successfully simulated it in Proteus. This lab provided practical experience in creating, building, and running an STM32 project, helping us understand the basic workflow of embedded systems development.

STM32CubeIDE Files Setup

- Core = Main Code
- Includes = Header files
- Drivers = STM32 libraries
- LAB2.ioc = Hardware configuration
- Binaries = Compiled output files
- Debug = Build and debug files
- FLASH.Id = Memory layout configuration



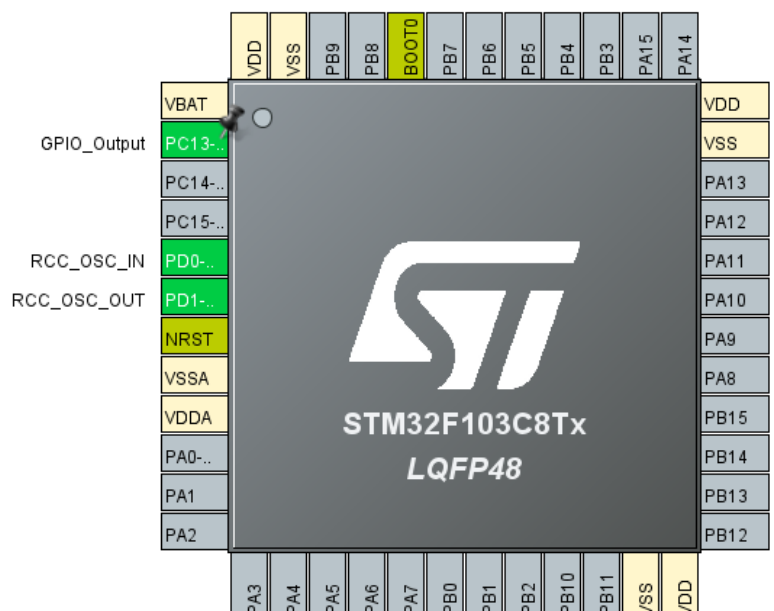
.ioc file

This is the STM32CubeMX configuration file.

It stores:

- Selected microcontroller
- Pin configuration
- Clock settings
- Peripheral settings
- Middleware configuration

Whenever you open this file, STM32CubeIDE launches the graphical CubeMX interface.



Blinky & Running LEDs C++ Using Proteus

Introduction

In this Task, we used C++ to program the STM32F103C8T6 microcontroller. We applied basic object-oriented programming (OOP) concepts such as constructors, destructors, and inheritance to organize the code in a structured and reusable way. We also learned how to create and use header (.h) files for better code management and modular programming. Finally, we successfully compiled and ran the program in Proteus to simulate the STM32 microcontroller and verify its operation.

GPIO

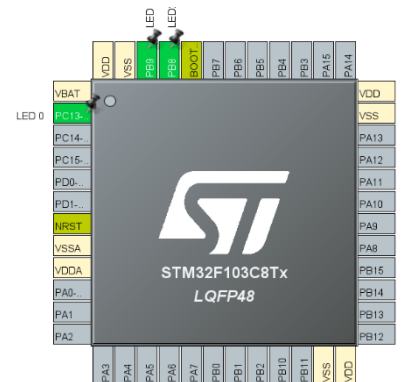
GPIO stands for General Purpose Input/Output.

It is a pin on a microcontroller that can be configured as either an input or an output.

When the pin is an INPUT state they can read electrical signal.

When the pin is in OUTPUT state they can send electrical signal.

In that case we use GPIO Pins PC13 (onboard LED),PB8,PB9 and State as GPIO_OUTPUT.



Search (Ctrl+F)

☐ Show only modified Pins

Pin Na...	Signal on ...	GPIO output I...	GPIO mode	GPIO Pull...	Maximum ...	User Label	Modified
PB8	n/a	Low	Output Pu...	No pull-up...	Low	LED2	✓
PB9	n/a	Low	Output Pu...	No pull-up...	Low	LED1	✓
PC13-TA...	n/a	Low	Output Pu...	No pull-up...	Low	LED 0	✓

CLOCK CONFIGURATION

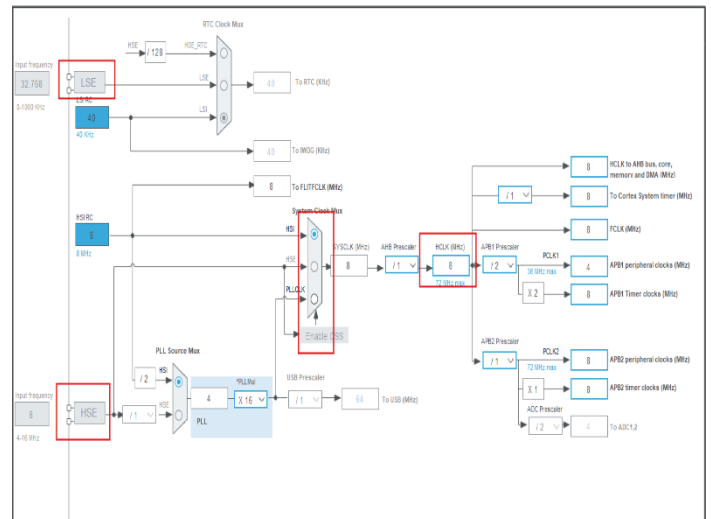
Clock Configuration is where you tell the STM32 microcontroller how fast it should run and which clock source it should use.

Types of Clock Sources

- High-Speed Internal (HSI)
- High-Speed External (HSE)
- Low-Speed Internal (LSI)
- Low-Speed External (LSE)

Quartz Crystal

A crystal (also called a quartz crystal) is an electronic component that generates a very accurate and stable clock signal for a microcontroller or other electronic devices.



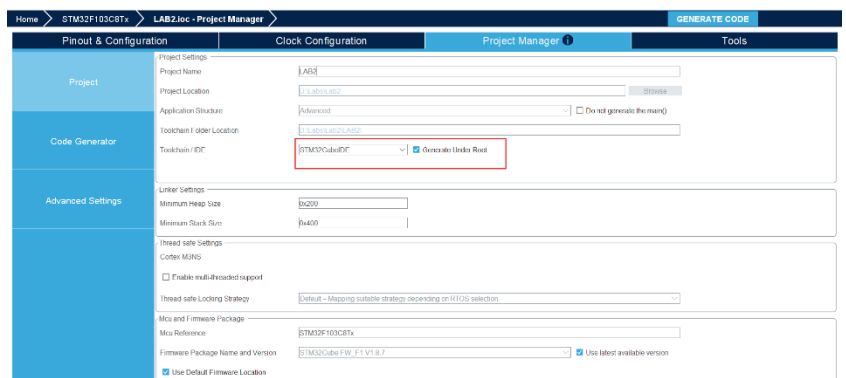
When an electric voltage is applied, the quartz crystal vibrates at a fixed frequency due to the piezoelectric effect. These vibrations are used to produce precise clock pulses.

Example

- 8 MHz Crystal → Used as the HSE (High-Speed External) clock.
- 32.768 kHz Crystal → Used as the LSE (Low-Speed External) clock for the Real-Time Clock (RTC).

How to Create a stm32cudeide file on STM32cubeMX

Go to the Project Manager tab and open the Tool/IDE settings. From the available options, select STM32CubeIDE as the development environment. After selecting it, click Generate Code to create the project files for STM32CubeIDE.



STM32CODEIDE C++ CODE

First create two files one is source and other is header and source file save on src folder and header file save on inc folder and file names are gpio_hal.h , gpio_hal.cpp and write C++ code in these files.

GPIO SYNTAX

GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState

- GPIOx specifies the GPIO port.

Examples: GPIOA, GPIOB, GPIOC

- GPIO_Pin specifies the pin number within the selected port.

Examples: GPIO_PIN_0, GPIO_PIN_5, GPIO_PIN_13.

- PinState specifies the logic level of the GPIO pin.

It determines whether the pin is set HIGH or LOW.

HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);

write() Function

The write() function is used to set the output state of a GPIO pin. It can drive the pin HIGH (GPIO_PIN_SET) or LOW (GPIO_PIN_RESET) depending on the value provided.

```
void Dout::write(GPIO_PinState state)
{
    HAL_GPIO_WritePin(_GPIOx, _GPIO_Pin, state=GPIO_PIN_SET);
}
```

toggle() Function

The toggle() function is used to change the current state of a GPIO pin. If the pin is HIGH, it becomes LOW; if it is LOW, it becomes HIGH.

```
void Dout::toggle(void)
{
    HAL_GPIO_TogglePin(_GPIOx, _GPIO_Pin);
}
```

Main.cpp

In main source file we call our header make functions and write conditions. In class Dout called Digital OUTPUT with public GPIO Class , Dout can call all GPIO Public functions.

In this program, the STM32 microcontroller is initialized using the HAL (Hardware Abstraction Layer) library, and the system clock is configured to ensure proper operation. Three LED objects (led0, led1, and led2) are created by assigning them to their respective GPIO ports and pins.

The program then enters an infinite while(1) loop, where each LED is toggled one after another. The toggle() function changes the LED's current state from ON to OFF or OFF to ON, while HAL_Delay(100) introduces a delay of 100 milliseconds between each toggle.

```
class GPIO
{
public:
    GPIO();
    ~GPIO();

    static void init(void);

private:
    static bool isInit;
};

class Dout : public GPIO
{
public:
    Dout(GPIO_TypeDef *GPIOx,
         uint16_t GPIO_Pin,
         GPIO_PinState state = GPIO_PIN_SET);

    ~Dout();

    void write(GPIO_PinState state);
    void toggle(void);
    GPIO_PinState read(void);

private:
    GPIO_TypeDef *_GPIOx;
    uint16_t _GPIO_Pin;
};
```

```
void SystemClock_Config(void);
int main(void)
{
    HAL_Init();

    SystemClock_Config();

    Dout led0(LED0_GPIO_Port, LED0_Pin);
    Dout led1(LED1_GPIO_Port, LED1_Pin);
    Dout led2(LED2_GPIO_Port, LED2_Pin);
    while (1)
    {
        /* USER CODE END WHILE */
        led0.toggle();
        HAL_Delay(100);

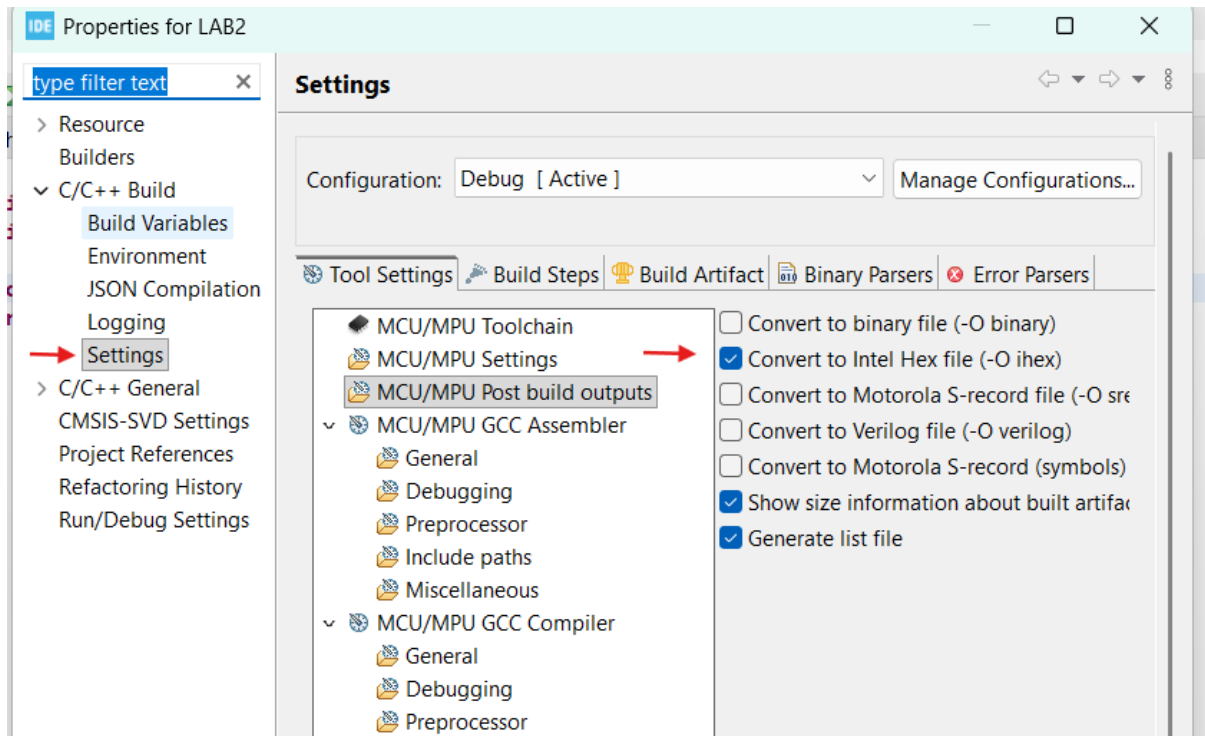
        led1.toggle();
        HAL_Delay(100);

        led2.toggle();
        HAL_Delay(100);
        /* USER CODE BEGIN 3 */
    }
    /* USER CODE END 3 */
}
```

How to Create a .hex File

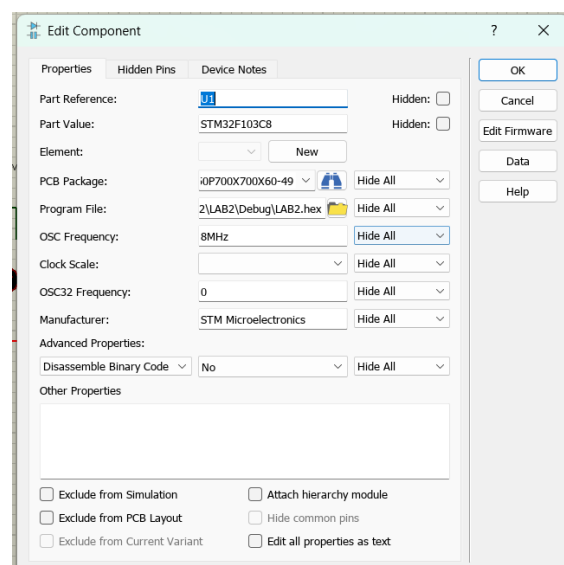
A .hex file is important when you want to check your code output on Design Software e.g. PROTEUS

Create a .hex file first click on project and go to the properties and then go to the C/C++ Build and then go to the settings and select .hex file create.



Proteus Software

After opening the Proteus select you design MCU and go to the properties and select to the program file Now you insert your create .hex file and save it on run the Program

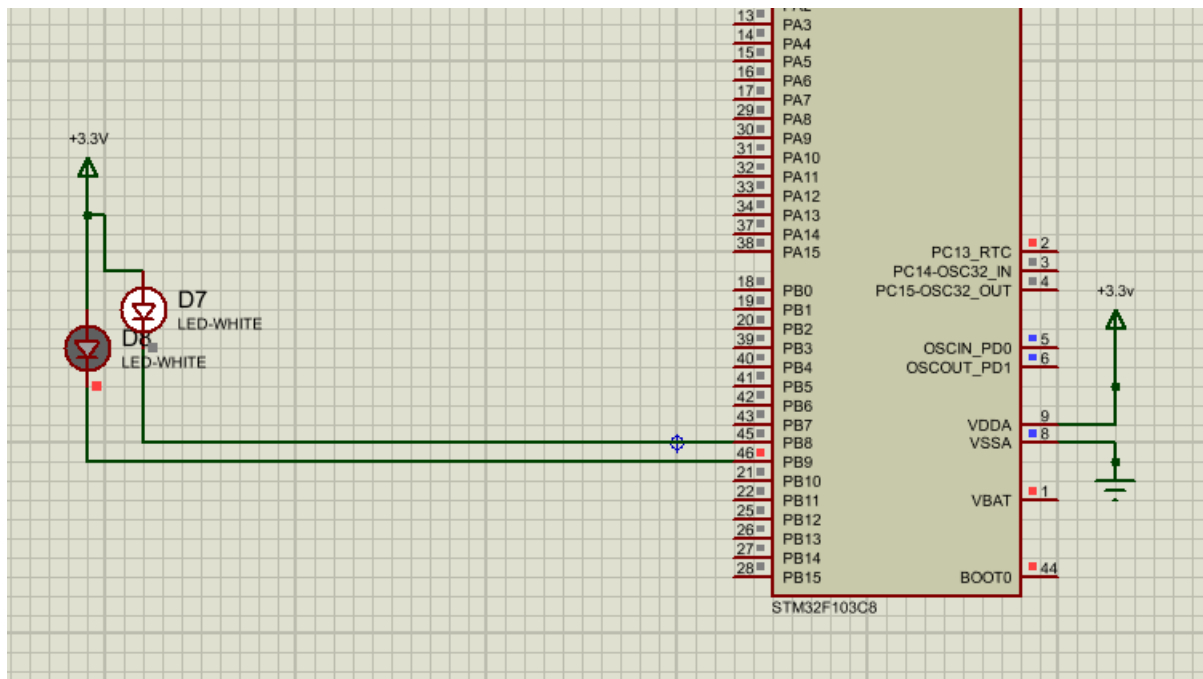


OUTPUT

Our output is the leds are toggle means on and off in sequence

HAL_Delay() Function:

In our code we set HAL_Delay(100) means Waits for 100 milliseconds before executing the next instruction.



- The program runs inside an infinite while(1) loop.
- LED0 blinks first, followed by LED1, and then LED2.
- Each LED changes its state after a 100 ms delay.
- This sequence repeats continuously, creating a running LED (chasing light) effect.